Solving the Lorenz ODE System Using Optimal ANN Architectures

By

Ikramul Hasan Moral (0112230489)

Md. Abu Bakar (0112230200)

Samiur Rahman Omlan (0112230195)

Fariha Islam (0112230431)

Md. Touhidul Islam (0112230435)

Submitted in partial fulfilment of the requirements
of the degree of Bachelor of Science in Computer Science and Engineering

June 13, 2026

Department of Computer Science and Engineering
United International University

Abstract

In physical modeling, nonlinear coupled ordinary differential equations (ODEs) are frequently encountered. It is not possible to obtain their analytical solutions. In most cases, the traditional approach to solving such problems is numerical methods such as the Runge-Kutta family, which comes with a computational burden and mesh-dependency. The Lorenz ODE system is a simplified meteorological model that is used as a test case in this study. This study investigates whether a standard artificial neural network (ANN), without physics-informed loss functions or operator learning configurations, can approximate the solution to this coupled system. Accurate reference data are generated using the Runge-Kutta method and the SciPy solver. Several feedforward ANN structures are then trained on this reference data. Then, change the network structure by adjusting the number of hidden layers, the number of neurons per layer, and the activation functions. Lastly, test the models by checking the mean squared error (MSE), training convergence, and the time required to compute them.

Acknowledgements

First of all thanks to ALMIGHTY ALLAH for giving us the strength and ability to complete this project successfully.

We sincerely express our gratitude to our honorable mentor, Dr. Muhammad Nomani Kabir, Professor, Department of CSE, United International University, for his guidance, support, and encouragement throughout this project.

We also thank our respected course teacher, Dr. Md. Motaharul Islam, Professor, Department of CSE, United International University, for his kindness and valuable teachings.

We are also grateful to all the faculty members who supported and guided us in completing this project.

Last but not the least, we thank our parents and family members for their constant support, love, and encouragement.

Table of Contents

List of Figures

List of Tables

Chapter 1

Introduction

This chapter introduces the research topic, provides an overview and motivation for the study, and outlines the main objectives and expected outcomes. It also outlines the methodology and briefly describes how the rest of the report is organized.

1.1 Project Overview

Ordinary Differential Equations (ODEs) describe how a system change with over time. They are widely used in fields such as physics, engineering, and mathematics. In many real-world problems, particularly those involving complex or nonlinear equations, it is often not possible to obtain exact solutions. That is why methods such as Euler's method and Runge-Kutta are used to obtain approximate solutions. In this project, we examine a particular system consisting of three nonlinear equations known as the Lorenz-1960 system (one of the first meteorological models). The system is small, but complex enough to make a good testing ground for various solution methods. The Lorenz-1960 system is a great candidate for evaluating how well artificial neural network (ANN) learn and predict behavior. Classical numerical methods address these problems step by step. Also they are typically accurate, but they are very slow and require more computational cost. We use numerical methods like Runge-Kutta to generate data that can be used to train artificial neural networks to learn how variables change over time. The ANN learns to match its predictions with the results of the numerical solver during training. After training, the ANN can quickly predict results for any time without solving the equations again. Therefore, the primary aim of this project is to solve the Lorenz-1960 system and to see how well a plain ANN can learn its behavior and which ANN architecture provides optimal results.

1.2 Motivation

This project is research and application driven. Classic numerical methods have been tested and proven to be accurate and reliable. They work well for directly simulating

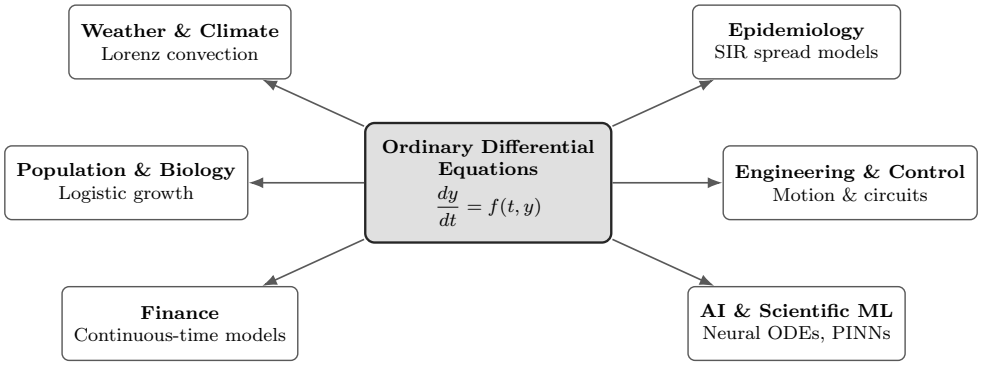

Figure 1.1: Ordinary differential equations describe rate-of-change processes throughout science, engineering, and modern machine learning. The Lorenz-1960 system studied in this report belongs to the weather and climate family.

equations. A trained artificial neural network (ANN), on the other hand, can solve the equation on its own. It forecasts and behaves as a copy of the actual system. Numerous investigations are currently used: Solving differential equations with neural network approximators based on PINNs, operator learning and ANN. This makes the topic interesting and relevant to investigate. Another motivation for this project is the learning and hands-on benefits it offers. It offers the building of skills in solving an ordinary differential equation(ODEs). It also provides training on how to construct and utilize models of the neural network. The project involves running experiments and analysing results. It's useful to compare different methods. Although simple ANNs are not as powerful as advanced methods, they will give us some insights into their abilities and limitations.

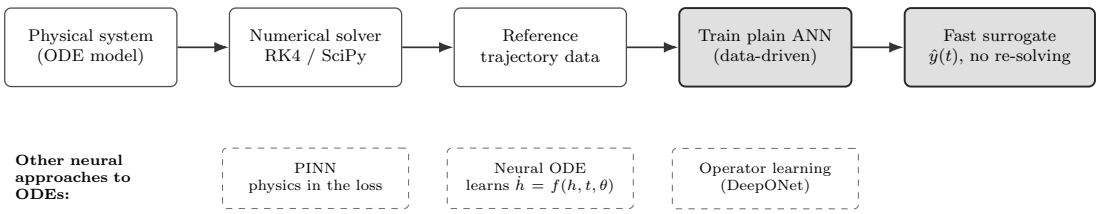

Figure 1.2: Ordinary differential equations underpin modern scientific machine learning. A numerical solver produces reference data that trains an artificial neural network; the trained network then acts as a fast surrogate that predicts the state at any time without re-solving the equations.

1.3 Objectives

The main aims of this project are:

- To numerically solve a coupled and nonlinear system (Lorenz equation).

- To apply ODE concepts, numerical methods such as the Runge-Kutta and SciPy solvers, The Lorenz equations are solved using an ANN. Numerical solutions obtained from the Runge-Kutta method.

- To generate accurate data with the Runge-Kutta method and then verify the numerical solutions with it. Compare and evaluate the performance of ANN.

- To compare the different ANN architecture to varying the number of layers, varying number of neurons per layer and testing various activation functions to determine the most optimum one.

- To evaluate the model's performance we are using mean squared error, learning stability, and other measures, we assessed the performance of the model. See if a plain ANN can learn the Lorenz equation and explore the trade off between the two variables, namely computational cost.

- To compare the results with PINN based methods from other research to understand how this approach performs compared to recent techniques.

1.4 Methodology

The "Waterfall Methodology" is being followed. Here, the various phases are carried out one after the other in a fixed order. It is fitting as we know what we want and need from the outset. In this, the input of each step is the output of the next step. This helps the process to be structured, easy to track and easy to manage.

In the Literature Review phase the Lorenz-1960 equation and its property was studied. Brief review of numerical methods, such as RK4, SciPy solvers. Besides, a number of ANN architectures, activation functions and training techniques were also investigated.

The problem of the study and the objectives of the study were well stated in the Concept Development phase. The configuration of the layers in the ANNs was focused on approximation of the Lorenz-1960 equation, and the number of neurons, activation functions, learning rates and epochs of training were selected. Data sets for training and validation were prepared.

During the Design phase, ANN models and training strategy were developed, such as data processing and optimization of the network structure. The RK4 and SciPy were used to calculate a numerical baseline to compare with. Assessment measures, visualization tools, and statistical tests were set up to guarantee a valid comparison of models. Some statistical tests were also outlined to ensure appropriate comparisons between models.

During the Implementation phase numerical solvers and ANN models were developed. All of the models were trained using the datasets generated. The accuracy, training time, parameters number and convergence was measured. Each model was performed systematically based on the design.

In the Testing phase, we then tested the performance of each ANN model and then predicted The results obtained by using the mean square error to evaluate its performance. We tested for generalization with new data. Compared the sensitivity of the models to various parameter values. The best models were compared with results from published studies.

Lastly, we have the Deployment phase, during which we incorporated all the results, plots, and analyses in the final project report. We designed and documented the code so that it was easy to repeat our work. The instructions for repeat experiments and retraining the models were included. We also presented our final suggestions on the best ANN architecture. This phase ensures that the project is complete and ready.

For better understanding, a diagram of our methodology is given below:-

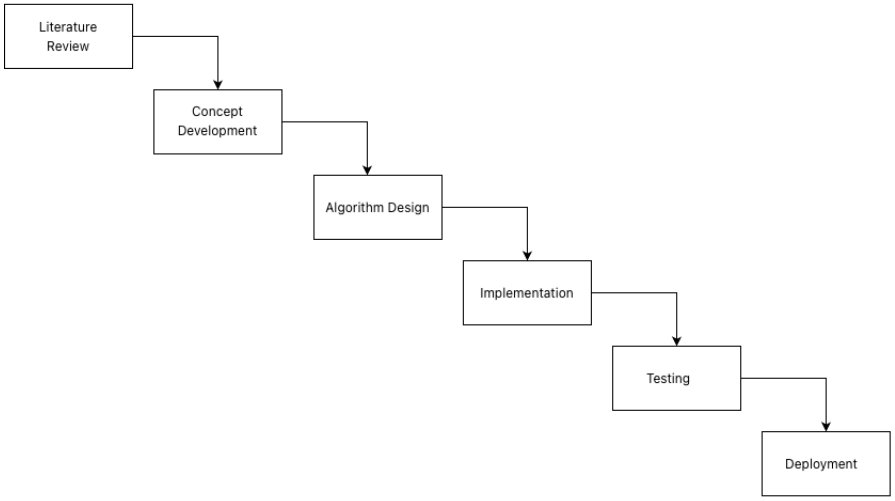

Figure 1.3: Methodology Diagram

1.5 Project Outcome

The expected outcome of the project will be the clear and the justified determination of the most appropriate architecture for the ANN in estimating the solution to the Lorenz-1960 system of ODEs within the scope of the experiment. This project must also include a numerical reference solution to the ODEs, the establishment of ANN baselines, and a comparative analysis of the results.

The project will also contribute to the current understanding of the capacity and limitations of ANN models in the approximation and solution of differential equations. If the ANN models are able to get a satisfactory level of accuracy in approximating the solution to the ODEs, the project will have demonstrated the capability of a simple ANN architecture to effectively act as an alternative numerical solver. This would suggest that a plain ANN, without the need for physics-informed loss terms or traditional mesh-dependent

numerical methods, can on its own approximate ODE solutions with sufficient accuracy while being computationally less expensive in terms of inference time and the avoidance of iterative re-solving for each query. If the models do not achieve satisfactory accuracy in approximating solutions to the ordinary differential equations (ODEs), the study will clearly delineate the limitations of simple ANN architectures and mention the necessity for more advanced methods.

This project will not only focus on coding and experiments, but also on understanding how different ANN structures, such as depth, width, and activation functions, influence the accuracy of solving coupled nonlinear ODE systems.

1.6 Organization of the Report

The rest of this report is outlined as follows. Chapter 2 outlines the theoretical background and literature review on Ordinary Differential Equations (ODE) solving, artificial neural networks, and existing neural methods for differential equations. Chapter 3 discusses about the project design, requirements, methodological plan, and task assignment. Chapter 4 outlines the implementation & experiment design used in this research. Chapter 5 outlines the results and analysis on the tested ANN structures. Finally, Chapter 6 concludes this report by focusing the main results, limitations, and potential directions for the future research.

Chapter 2

Background

This chapter covers the foundational knowledge you need to understand our work. We start with the mathematical and computational preliminaries. Then we review the existing literature on ANN-based ODE solvers and similar applications. We identify the research gaps that motivate our study. Finally, we wrap up with a brief summary.

2.1 Preliminaries

This section provides the necessary background knowledge to understand the rest of the report.

2.1.1 Ordinary Differential Equations

An ODE describes how a quantity changes with respect to a single independent variable. A general first-order ODE takes the form:

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0 \tag{2.1}$$

where $y(t)$ is the unknown function, t is the independent variable, and y_0 is the initial condition.

2.1.2 The Lorenz-1960 System

The Lorenz-1960 system [3] consists of three coupled nonlinear ODEs for meteorological modeling. We target the formulation from [4] (Section 4.2):

$$\begin{aligned}
\frac{dx}{dt} &= kl\left(\frac{1}{k^2+l^2} - \frac{1}{k^2}\right) yz \\
\frac{dy}{dt} &= kl\left(\frac{1}{l^2} - \frac{1}{k^2+l^2}\right) xz \\
\frac{dz}{dt} &= \frac{kl^2}{2}\left(\frac{1}{k^2} - \frac{1}{l^2}\right) xy
\end{aligned} \tag{2.2}$$

with $k = 2$, $l = 1$, initial conditions $x(0) = 0.5$, $y(0) = 0.75$, $z(0) = 1$, and $t \in [0, 1]$. Its nonlinear, coupled dynamics make it an excellent benchmark for numerical solvers.

2.1.3 Classical Numerical Methods

Euler's method approximates the solution via a first-order Taylor expansion:

$$y_{n+1} = y_n + h \cdot f(t_n, y_n) \tag{2.3}$$

where h is the step size. It is simple but accumulates errors quickly.

The fourth-order Runge-Kutta method (RK4) evaluates the derivative at multiple points per step for higher accuracy:

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \tag{2.4}$$

Both methods are mesh-dependent, require re-solving for every new initial condition, and struggle with stiff equations.

2.1.4 Artificial Neural Networks

A feedforward ANN (Multilayer Perceptron) consists of an input layer, hidden layers, and an output layer. Each neuron computes:

$$a_i(\mathbf{x}) = \sigma\left(\mathbf{w}^{(i)} \cdot \mathbf{x} + b^{(i)}\right) \tag{2.5}$$

where $\mathbf{w}^{(i)}$ are weights, $b^{(i)}$ is bias, and σ is an activation function [5]. The Universal Approximation Theorem [5, 6, 7] guarantees that a network with at least one hidden layer can approximate any continuous function to arbitrary accuracy. Common activation functions include:

- **Sigmoid**: $\sigma(x) = \frac{1}{1+e^{-x}}$, maps inputs to $(0, 1)$
- **Tanh**: $\sigma(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$, maps inputs to $(-1, 1)$
- **ReLU**: $\sigma(x) = \max(0, x)$, efficient but non-smooth at zero
- **GELU/Swish**: smooth ReLU approximations with better gradient flow

Training minimizes a loss function $L(\theta)$ over network parameters θ using gradient descent. Backpropagation computes gradients via the chain rule, and optimizers like Adam adapt learning rates per parameter [8, 9].

2.1.5 ANN-Based ODE Solving

Instead of stepping through the domain, ANNs learn a continuous solution function over the entire domain. The standard approach [10, 11, 12] constructs a trial solution:

$$\psi_t(x, \mathbf{p}) = A(x) + F(x, N(x, \mathbf{p})) \tag{2.6}$$

where $A(x)$ satisfies the initial/boundary conditions exactly, and $N(x, \mathbf{p})$ is the neural network output. The loss function measures how well this trial solution satisfies the ODE:

$$L(\mathbf{p}) = \frac{1}{N} \sum_{i=1}^{N} \left[\frac{d\psi_t(x_i, \mathbf{p})}{dx} - f(x_i, \psi_t(x_i, \mathbf{p})) \right]^2 \tag{2.7}$$

Minimizing this loss yields a differentiable, closed-form approximate solution without requiring knowledge of the exact answer.

2.1.6 Physics-Informed Neural Networks

PINNs embed governing equations directly into the loss function [4, 6, 9, 13]:

$$L(\theta) = L_{\text{data}} + \lambda_1 L_{\text{physics}} + \lambda_2 L_{\text{IC/BC}} \tag{2.8}$$

The physics loss uses automatic differentiation to compute derivatives of the network output. This enables training with sparse or no data, but balancing the loss terms requires careful hyperparameter tuning.

2.1.7 Network Architectures

To better understand the differences in methodology, we present the standard architectures used in recent studies. Figure 2.1 illustrates a typical Artificial Neural Network (ANN) architecture used for predicting system parameters without physics-informed constraints [1]. This standard feedforward structure relies entirely on data-driven learning through backpropagation.

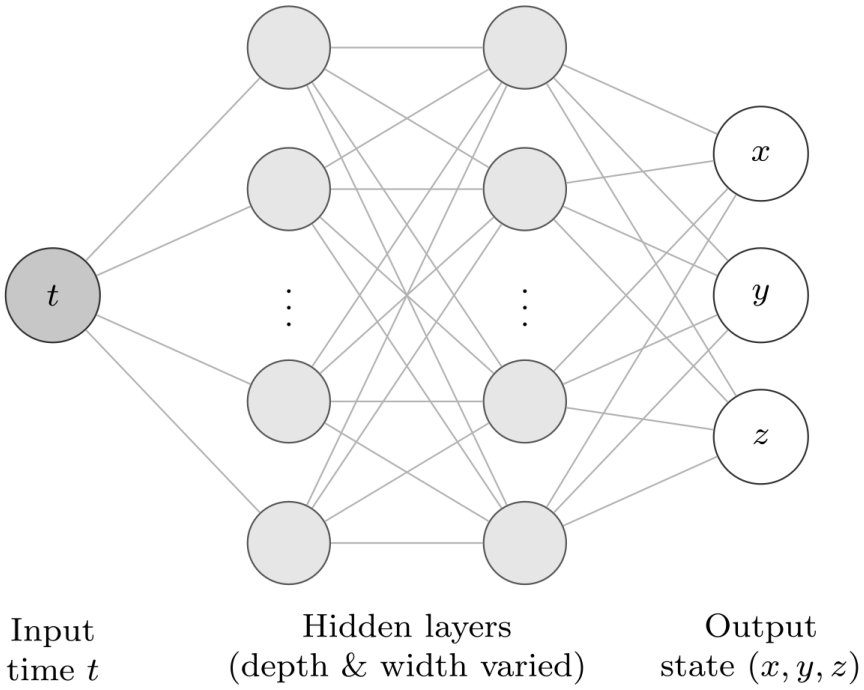

Figure 2.1: Typical Artificial Neural Network architecture [1].

In contrast, Figure 2.2 demonstrates a Physics-Informed Neural Network (PINN) architecture. In this approach, the standard ANN output is further constrained by physical laws—specifically, the governing differential equations of the system—which are embedded directly into the loss function [2].

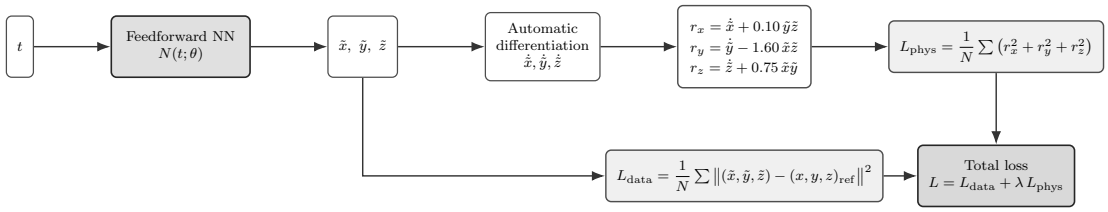

Figure 2.2: Architecture of a Physics-Informed Neural Network incorporating physical loss [2].

2.2 Literature Review

2.2.1 Similar Applications

Dubey et al. [8] used an ANN (tanh activation) to solve coupled first-order ODEs with an error of 2.88×10^{-3} in a maximum of 120 iterations. A comparable PyTorch-based ANN was used by Audu et al. [14], which converged in 200–1500 iterations. Okereke et al. [12]

bypassed backpropagation by calculating the weights based on regression with Gaussian Radial Basis Functions. The ODEN framework developed by Lau and Werth [5] demonstrates that NNs outperform RK4 on rapidly oscillating solutions, with errors of $\sim 10^{-4}$. On a damped vibration equation, Mall and Chakraverty [11] compared arbitrary versus regression-based weight initialization and they found that regression-based weights reduce maximum deviation from 3.67% to 1.47%. Murugesh et al. [15] used neural networks with block numerical methods to directly solve higher-order ODEs and obtaining errors near machine precision ($\sim 10^{-10}$). Pratama et al. [16] surveyed three ANN-based PDE solvers (PyDEns, NeuroDiffEq, Nangs) and used 3 hidden layers with 32 neurons and tanh activation. NeuroDiffEq uses the Trial Approximate Solution methodology, which suggests that initial conditions are hardcoded in the ansatz. This is a pure ANN method that is applied to our research.

2.2.2 Related Research

Physics-Informed Approaches

PinnDE was proposed by Matthews and Bihlo [4], which solved the Lorenz-1960 system using a DeepONet (4 hidden layers, 40 nodes, tanh, Adam optimizer). Their DeepONet is trained to learn the operator $G(u_0)(t, x)$ which provides solutions given initial conditions over $[0, 1]$. This is the closest baseline to our target problem, but it relies on physics-informed loss terms. Babni et al. [6] generalized the trial construction $\tilde{u}(x, \theta) = P(x) + Q(x)N(x, \theta)$ by replacing $Q(x)$ with a flexible $\varphi(x)$, proving the error bound $|N_\varphi(t, \theta) - u(t)| \leq e^{\kappa|t-t_0|}L_\varphi(\theta)$. This verifies that the smaller the loss, the smaller the solution error. Audu et al. [9] compared PINN and ANN models using six first-order ODE tests. Their PINN had an error of 10^{-8} to 10^{-10} which was much better as compared to ANN. This raises the issue of whether architecture optimization can bridge this gap or not.

Pure ANN Approaches

The NeuroDiffEq Trial Approximate Solution is physics-free. Pratama et al. [16] found that NeuroDiffEq can train without physics loss terms which is consistent with our methodology. But they benchmarked only on the heat equation which is linear. Dubey et al. [8] claim that plain ANNs can solve coupled ODEs but used a fixed architecture. Okereke et al. [12] removed backpropagation through weight computation instead of polynomial fitting. Audu et al. [14] achieved an error of 10^{-4} with a sigmoid ANN on three first-order ODEs. All these studies do not examine the implications of building patterns on performance.

Architecture and Loss Function Studies

Mall and Chakraverty [11] made comparisons between 4, 5, and 6 hidden nodes with random and regression-based weights. Weights that are based on regression are always better

compared to random weights. After 6 nodes they found that accuracy remains the same. They applied only sigmoid activation on simple ODEs. Lau and Werth [5] discovered that sigmoid / tanh need less training in comparison to ReLU / ELU. Xavier initialization speeds up the convergence and Adam is better than SGD. They did not vary systematically depth and width. Xiong [17] arranged seven constructions (polynomial, exponential, hyperbolic, logarithmic, logistic, sigmoid, softplus) of loss functions. Exponential is best when applied to the law of Newton cooling and logarithmic is best when it is applied to the motor suspension system.

Lorenz System Studies

Aslam et al. [18] have used ANN + PSO-NNA hybrid optimization to solve Lorenz equations (Neurons=10, sigmoid, N=90, independent runs=100). They are directed to a diverse Lorenz parameterization (σ, ρ, β), are based on meta heuristic optimization and do not compare architectures.

Neural ODEs

Chen et al. [19] proposed neural ODEs where the derivative function $\frac{dh(t)}{dt} = f(h(t), t, \theta)$ that is parameterized with a neural network and trained using the adjoint method efficient for backpropagation. This sample is quite different than ours where we learn the dynamics (f) while neural ODEs find the approximation of the solution $(y(t))$ itself.

Hybrid Approaches

Murugesh et al. [15] proposed a Neural-ODE Hybrid Block Method with the update rule:

$$y(t_{k+1}) = y(t_k) + h \sum_{j=0}^{m-1} \omega_j f_\theta(t_{k-j}, y_{k-j}) + \sum_{i=0}^{m-1} \alpha_i F(t_{k-i}, y_{k-i}) \tag{2.9}$$

where ω_j and α_j balance neural versus numerical contributions. It achieved machine-precision errors on harmonic oscillators and the Van der Pol equation.

2.3 Gap Analysis

Table 2.1 summarizes what each reviewed paper covers and where gaps exist.

Table 2.1: Comprehensive Literature Gap Analysis

Paper	**Methodological Approach**	**Target Differential Equations**	**Architectural Investigation**	**Lorenz-1960 Application**	**Plain ANN Implementation**
[4]	PINN + DeepONet: Integrates physics laws and operator networks into the loss function.	Solves both ODEs and PDEs using governing physics.	No systematic study of network structural impact.	Evaluated specifically as a complex dynamical system.	Uses physics-informed logic, not a plain ANN.
[16]	PyDEns / ANN: Standard networks for field estimation.	Primary focus on solving Partial Differential Equations (PDEs).	General overview without specific layer/neuron testing.	The study does not address the Lorenz model.	Covers standard ANN architectures for PDEs.
[19]	Neural ODE: Derivative represented as a neural network.	Models continuous-time dynamics for ODEs.	Focuses on functional flexibility, not structural depth.	Not specifically tested on the Lorenz system.	Relies on specialized ODE-solver integration.
[18]	ANN + PSO-NNA: ANN with Particle Swarm Optimization.	Applied to solving Ordinary Differential Equations.	Optimization focuses on weights, not architectural depth.	Directly applied to Lorenz-based chaotic dynamics.	Uses heuristic optimization instead of plain training.
[5]	Plain ANN: Standard feedforward network approach.	General application for approximating ODE solutions.	Provides a limited (partial) study on hidden layers.	Tested on other benchmarks, not Lorenz-1960.	Utilizes a standard, plain ANN configuration.
[11]	ANN: Implementation of neural networks for equations.	Targets solutions for ODEs in engineering.	Includes a detailed study on network architecture.	Application restricted to simpler, non-chaotic ODEs.	Employs a basic feedforward ANN structure.
[17]	ANN: Simple neural solver for dynamic systems.	Focused on solving Ordinary Differential Equations.	Does not explore the impact of different layer counts.	Does not address the chaotic Lorenz-1960 model.	Uses a traditional, data-driven ANN model.
[6]	PINN: Physics-Informed Neural Networks approach.	Solves ODEs by embedding equations in the loss function.	No investigation into depth or width performance.	Not applied to the meteorological Lorenz model.	Strictly uses physics-informed constraints.
[8]	Plain ANN: Feedforward network trained on numerical data.	Applied to various Ordinary Differential Equations.	Lacks a systematic study of network architecture.	No specific testing on the Lorenz system.	Uses a standard numerical data training pipeline.
[12]	ANN: Basic neural network models for ODEs.	Aimed at solving standard ODE systems.	Does not perform structural performance analysis.	No mention or testing of the Lorenz-1960 system.	Follows a traditional, plain ANN approach.
[14]	Plain ANN: Usage for initial value problems in ODEs.	Specifically handles ODE solution approximation.	No detailed evaluation of layers or neurons.	Focused on other common types of ODEs.	Relies on a standard feedforward structure.
[9]	PINN vs ANN: A comparative study of two methods.	Analysis restricted to Ordinary Differential Equations.	Lacks an in-depth architecture comparison study.	Not applied to the Lorenz-1960 model.	Discusses ANN only in comparison to PINN.
[15]	Hybrid: Combines different neural modeling techniques.	Solves ODEs using integrated hybrid approaches.	Architecture optimization is not the primary focus.	Does not utilize the Lorenz-1960 test case.	Incorporates external optimization methods.
[10]	ANN: Trial-solution method using a feedforward network.	Solves ODEs, ODE systems, and PDEs.	No systematic study of layers or neurons.	Does not test the Lorenz-1960 system.	Plain feedforward ANN trained on the ODE residual.
[13]	PINN: Embeds governing equations in the loss function.	Targets forward and inverse nonlinear PDEs.	No systematic study of depth or width.	Not applied to the Lorenz-1960 model.	Uses physics-informed constraints, not a plain ANN.
[3]	Analytical: Reduces the vorticity equation to three ODEs.	Derives the coupled nonlinear Lorenz-1960 ODEs.	Predates neural network architecture studies.	Defines the original Lorenz-1960 system.	Uses no neural network of any kind.
[7]	Theory: Proves the universal approximation property.	Not concerned with differential equations.	Shows one hidden layer suffices for approximation.	Does not address the Lorenz-1960 system.	Covers feedforward networks in general, not a solver.
Our Work	**Plain ANN: Traditional feedforward network.**	**Direct solution of coupled ODE systems.**	**Comprehensive study of hidden layers and density.**	**Primary test case for predicting chaotic dynamics.**	**Purely data-driven ANN without physics aids.**

The literature review reveals five specific gaps that our research addresses:

1. **No architecture study for the Lorenz-1960 system:** Matthews and Bihlo [4] solve the Lorenz-1960 system but use a fixed DeepONet architecture without optimization. Aslam et al. [18] study a different Lorenz parameterization. No existing paper systematically compares ANN architectures on the Lorenz-1960 system.

2. **No PINN vs. plain ANN comparison on Lorenz-1960:** Audu et al. [9] compared PINN and ANN on simple first-order ODEs. No study makes this comparison on a nonlinear, coupled ODE system like Lorenz-1960.

3. **No systematic activation function study for ODE solving:** Pratama et al. [16] use tanh throughout without ablation. Lau and Werth [5] note that sigmoid and tanh outperform ReLU but do not test modern alternatives like GELU or Swish. Mall and Chakraverty [11] use only sigmoid. No study compares activation functions (tanh, ReLU, sigmoid, GELU, Swish) on a coupled ODE system.

4. **No systematic depth and width study for ODE solution quality:** Mall and Chakraverty [11] test 4, 5, and 6 nodes on simple ODEs. Lau and Werth [5] observe that architecture should match equation complexity. But no paper systematically varies both depth (2–6 layers) and width (20–100 neurons) to find the optimal configuration for a Lorenz-type system.

5. **Limited generalization studies:** Most papers train for a single initial condition. The ability to generalize across different initial conditions remains largely unexplored for coupled nonlinear ODE systems.

Our research fills these gaps by performing a systematic architecture comparison on the Lorenz-1960 ODE system using plain ANNs. We vary depth, width, and activation functions while benchmarking against RK4 reference solutions and the PINN-based results from Matthews and Bihlo [4].

2.4 Summary

This chapter established the necessary background for our research. We reviewed the mathematical foundations of ODEs, the Lorenz-1960 system, classical numerical methods, and ANN-based solving approaches. The literature review covered 17 research papers spanning physics-informed methods, pure ANN approaches, architecture studies, Lorenz system solvers, Neural ODEs, and hybrid techniques.

The gap analysis reveals that no existing study performs a systematic architecture comparison of plain ANNs on the Lorenz-1960 ODE system. Existing works either use fixed architectures, focus on simpler equations, or rely on physics-informed loss terms. Our research directly addresses this gap by varying network depth, width, and activation functions to identify the optimal ANN architecture for solving the Lorenz-1960 system as an initial value problem.

The next chapter presents our project design, including the detailed methodology, experimental setup, and evaluation plan.

Chapter 3

Project Design

In this chapter, we describe the design of the experiment. This chapter will be the requirements of our pipeline needs to meet also the context it operates in. The two-phase methodology used for the architecture search and optimizer comparison.

3.1 Requirement Analysis

We have a clear set of requirements for our pipeline. It should have certain outputs and has defined quality. Set goals and work within a set of limits. This is a computing research project, Not a software product is. Thus, the requirements are towards reproducibility, numerical accuracy, and disciplined experimentation.

3.1.1 Functional and Nonfunctional Requirements

The functional requirements describe the actions that the pipeline performs in each phase. Beginning at the pipeline's creation the reference solution to producing the final comparison report. The nonfunctional requirements determine the quality level. There are two strict requirements – reproducibility and reference accuracy.

Table 3.1: Functional Requirements of the Research Pipeline

ID	**Requirement**
FR1	The system shall be required to generate a high-accuracy numerical reference solution for The Lorenz-1960 ODE system using the fourth-order Runge-Kutta and the SciPy library, evaluated over the interval $t \in [0, 1]$ at 1001 uniformly spaced points.
FR2	The system shall partition the reference dataset into training and validation subsets using 80/20 random split with a fixed random seed.
FR3	The system shall construct feedforward ANN models with systematically varied parameters. These parameters include depth (1 to 4 hidden layers), width (20, 50, or 100 neurons per layer), and activation function (tanh, ReLU, sigmoid, GELU, or Swish).
FR4	The system shall train all Phase 1 models under identical conditions using the Adam optimizer and record convergence behaviour for each run.
FR5	The system shall conduct a Phase 2 optimizer comparison using the three best Phase 1 architectures. Each architecture will be trained with Adam, L-BFGS, and SGD with mo momentum.mentum. momentum.
FR6	The system shall evaluate each trained model using MSE, MAE, maximum absolute error, training time, and inference time against the RK4 reference.
FR7	The system shall produce comparison tables, solution trajectory plots, and error curves for every evaluated models.

Table 3.2: Nonfunctional Requirements of the Research Pipeline

ID	**Requirement**
NFR1	All experiments shall be fully reproducible using a single fixed random seed applied to weight initialization and data splitting.
NFR2	The numerical reference solution shall maintain solver agreement of RMSE below 10^{-10} between the custom RK4 implementation and SciPy.
NFR3	All results, model configurations, and metrics shall be logged systematically so that an independent reader can verify the reported findings.

3.1.2 Context Diagram

Figure 3.1 shows the research pipeline as one bounded system with two external entities. The first is a research team. We set up experiments and start them as well as gather the results. the resulting models and metrics. The second are the academic and research community. They eat the reported results, repeat the approach and extend the comparisons. results. Each internal process such as reference generation, final reporting, and so on resides within the pipeline. There are no other living processes outside the system boundary.

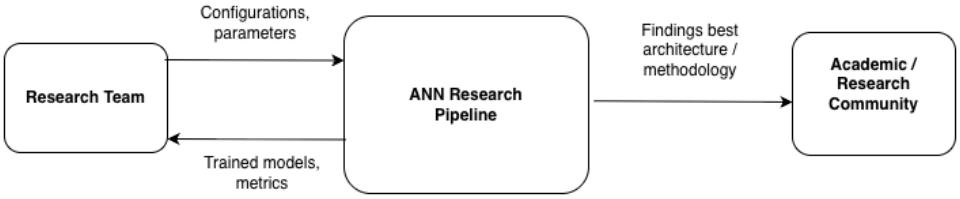

Figure 3.1: Context diagram of the research pipeline.

3.1.3 Data Flow Diagram Level 1

Figure 3.2 divides the research pipeline into a series of five steps. Process P1 generates Plot the reference trajectory with both the RK4- and SciPy-solvers. P2 makes it pre-process the trajectory It is done through the 80/20 split and z-score normalization of the output targets. P3 trains the ANN models, initially over the complete Phase 1 grid, and subsequently over the Phase 2 optimizer comparison. P4 test each of the trained models with the reference solution using the metrics from FR6. P5 compares the results, selects the best architecture and Creates the final report.

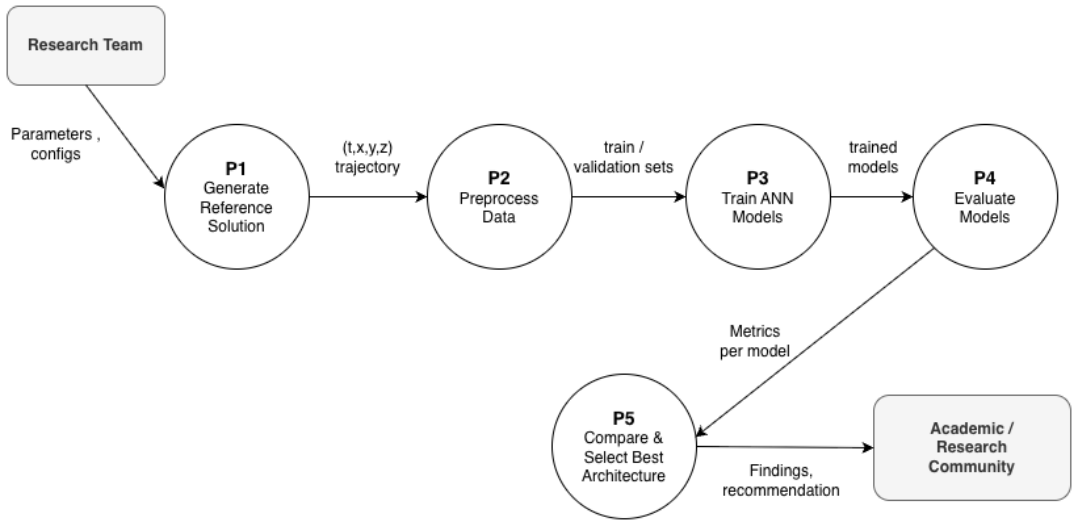

Figure 3.2: Data Flow Diagram (Level 1) showing the five internal processes of the research pipeline.

3.1.4 UI Design

This project has no user interface. We run everything from Python scripts and Jupyter notebooks. Results are stored as figures, tables, and serialized model files. UI design does not apply here.

3.2 Detailed Methodology and Design

The methodology is divided into two phases. Here is the logic.

- **Phase 1 – Architecture Search (60 runs):** We choose the depth: 1–4 hidden layers, width: 20, 50, 100 neurons, and activation functions: tanh, ReLU, Sigmoid, GELU and Swish. Also we choose the optimizer fixed at Adam. This will show us which architectural choices matter most.
- **Phase 2 – Optimizer Comparison (9 runs):** We take the three best architectures from Phase 1. Then fine-tune each one with the following three algorithms: Adam, L-BFGS, SGD with momentum. All other factors remain unchanged.All other factors remain constant. This indicates the percentage of optimization that the optimizer will use to contributes on top of the architecture.

This separation allows that the two comparisons are not confused. If any Improvements in Phase 2 do not result from optimizing the network, but from a change in the network structure.

3.2.1 Reference Solution Generation

We calculate the reference solution from the Lorenz-1960 system with $k = 2$, $l = 1$, initial conditions $x(0) = 0.5$, $y(0) = 0.75$, $z(0) = 1$, and time interval $t \in [0, 1]$, as defined in Chapter 2. Two independent solvers are used to ensure that reference numbers are accurate. The first is a custom-made fourth-order Runge-Kutta integrator with step size $h = 10^{-3}$. Evaluated at a common grid of points 1001 points. The second one is the solve ivp of SciPy routine using DOP853 with rtol = 10^{-10} and atol = 10^{-12} which is sampled on the same grid. We need agreement between both solvers at RMSE below 10^{-10} and then we use The RK4 trajectory was adopted as the reference trajectory for training and evaluation of all the ANNs. This dual solver are make sure to check and catch the unintentional mistakes in either implementation. Also it provides us a reliable reference and the rest of the study will be based on this.

3.2.2 Dataset Preparation

The reference trajectory provides us with 1001 ,$(t, x(t), y(t), z(t))$. That is our training set. Our training set is 80% training (around 800 points) and our validation set is 20% validation (about 201 points) and we report our results on the validation set. The data is split randomly into two halves, with seed 42. It comes as a surprise to us all but we do the split once before training any model. All configurations of the ANN use the same training points, validation points, and validation error.The same training points, the same validation points, and the same validation error are used in every ANN configuration. and in the same order. We then normalize each of the three target variables by using the training-set, the mean and standard deviation. Given x , y and z , this z score normalization puts x , y and z on a comparable scale, to ensure that the loss function is not being swamped by the variable with the largest range of variation range. The key to

this is that all errors are reported after unnormalisation. The published metrics stay in the original physical units of the system.

3.2.3 Model Family and Architecture (Phase 1)

Each candidate ANN is a simple feedforward network. Accepts scalar time t as an argument, it is used for predicting the three dimentional state vector $(x(t), y(t), z(t))$. The hidden structure varies with The number of hidden layers, the number of neurons in each hidden layer, activation function. We explore these values in Phase 1 as listed in Table 3.3.

The Cartesian product of these axes results in $4 \times 3 \times 5 = 60$ distinct architectures. We Optimize each one separately with the same optimization parameters. The output layer is always linear with three units. We do not include any skip connections, normalization layers, and no regularization. This is a simple design on purpose. It helps hold the comparison to one's mind. From our research question about the three architectural axes, the following questions were developed.

Table 3.3: Phase 1 Architecture Search Grid

Axis	**Values**
Depth (hidden layers)	1, 2, 3, 4
Width (neurons per layer)	20, 50, 100
Activation function	tanh, ReLU, sigmoid, GELU, Swish
Total configurations	$4 \times 3 \times 5 = 60$

3.2.4 Optimizer Comparison (Phase 2)

Phase 2 takes the three architectures with the lowest combined validation MSE from Phase 1 and retrains each one with two additional optimizers: L-BFGS and SGD with momentum. Everything else stays the same as Phase 1. We reuse the Adam result from Phase 1 as the third comparison point. That gives us $3 \times 3 = 9$ Phase 2 runs.

Why these optimizers? L-BFGS is a strong baseline for smooth, low-dimensional regression problems. Researchers commonly cite it as effective for ANN-based ODE solvers. SGD with momentum serves as a classical reference. This phase isolates the optimizer's effect from the network architecture's effect. This separation would not be achievable if optimizer selection were included in the Phase 1 search grid.

3.2.5 Loss Function and Training Protocol

The training loss is the mean squared error between the normalized predicted state and the normalized reference state, computed jointly over all three output components. We add no initial-condition penalty and no physics-based residual. This is consistent with the plain-ANN scope from Chapter 2 and avoids the loss-balancing problem that plagues physics-informed approaches.

Here are the training settings:

- Phase 1 uses Adam with learning rate 10^{-3} and default momentum parameters.
- Each model trains for at most 10,000 epochs.
- Early stopping monitors validation loss with a patience of 500 epochs and restores the best-validation weights when training ends.
- We use full-batch gradient descent. The training set is small enough that full-batch is appropriate, and it removes minibatch noise as a confounding variable across configurations.
- We set the PyTorch random seed to 42 before constructing each model, so weight initialization is reproducible too.

3.2.6 Evaluation Metrics

The results of each model trained are tested and compared with the RK4 reference result. using 5 metrics. Table 3.4 lists them. MSE is the major indicator as it corresponds to This training goal and allows them to compare all the configurations directly. The absolute error and the maximum absolute error are the reported per state variable. They provide a more comprehensible representation of the best and worst scenarios deviations. Training time relates to the amount of the cost to train a particular architecture. Inference time represents the cost of prediction of the complete trajectory when it is trained. Together, These five are the indicators that will help us in our search of the accuracy/cost trade-off we're looking for our research question.

Table 3.4: Evaluation Metrics Used for Each Trained Model

Metric	**Purpose**
MSE (per variable and combined)	Primary loss. Consistent comparison across all 69 runs
MAE (per variable)	Interpretable error magnitude in original physical units
Maximum absolute error	Worst-case deviation along the trajectory
Training time (seconds)	Computational cost of fitting a given architecture
Inference time (seconds)	Cost of predicting the full 1001-point trajectory after training

3.2.7 Alternative Solutions Considered

We looked at several alternatives before settling on the plain feedforward ANN with a pure MSE loss. Here is what we rejected and why.

A PINN would re-use the hyperparameter and add a residue term to the loss.Balancing problem described in Chapter 2. It also would make it difficult to distinguish between our work The current PinnDE research and the ongoing PinnDE study. So we dropped it.

The same reason was rejected for a DeepONet operator-learning approach. PinnDE A DeepONet is already tested by already on the same equation. The reproduction of that design would not address the architecture-selection gap that we found.

We explored the possibility of a Neural ODE formulation as it is a different paradigm. However, Neural ODEs do not parameterize the solution function, they parameterize the derivative. That does not give an answer to our research question

Lastly, we considered fusing the architecture search and an optimizer selection as a single process. We did not accept this as there would be more experiments in total. And mix the two effects we wish to separate from each other. The two phase design while still maintaining a clean attribution of the, adopted has the number of runs at 69 observed differences.

3.3 Project Plan

The project plan is a research design that is designed to be carried out over 42 weeks in the following way: FYDP-I, FYDP-II, and FYDP-III. The schedule groups the work into research and writing, baseline and data preparation, implementation, analysis and validation, presentation and The timeline is based on the sequence of submission milestones, according to the scope of the problem. Results verification, report, framing and numerical baseline validation to ANN experiments Completing and defence preparation. Each of the three phases are 14 weeks long, with the last one being a deliberate review. 2 weeks will be dedicated to each phase of the defense preparation and final presentation. The task placement is provided in Figure 3.3 and Table 3.5 shows the placement across the three 14-week trimesters. Specifies the names in the chart.

Figure 3.3: 42-week FYDP Gantt chart using task IDs.

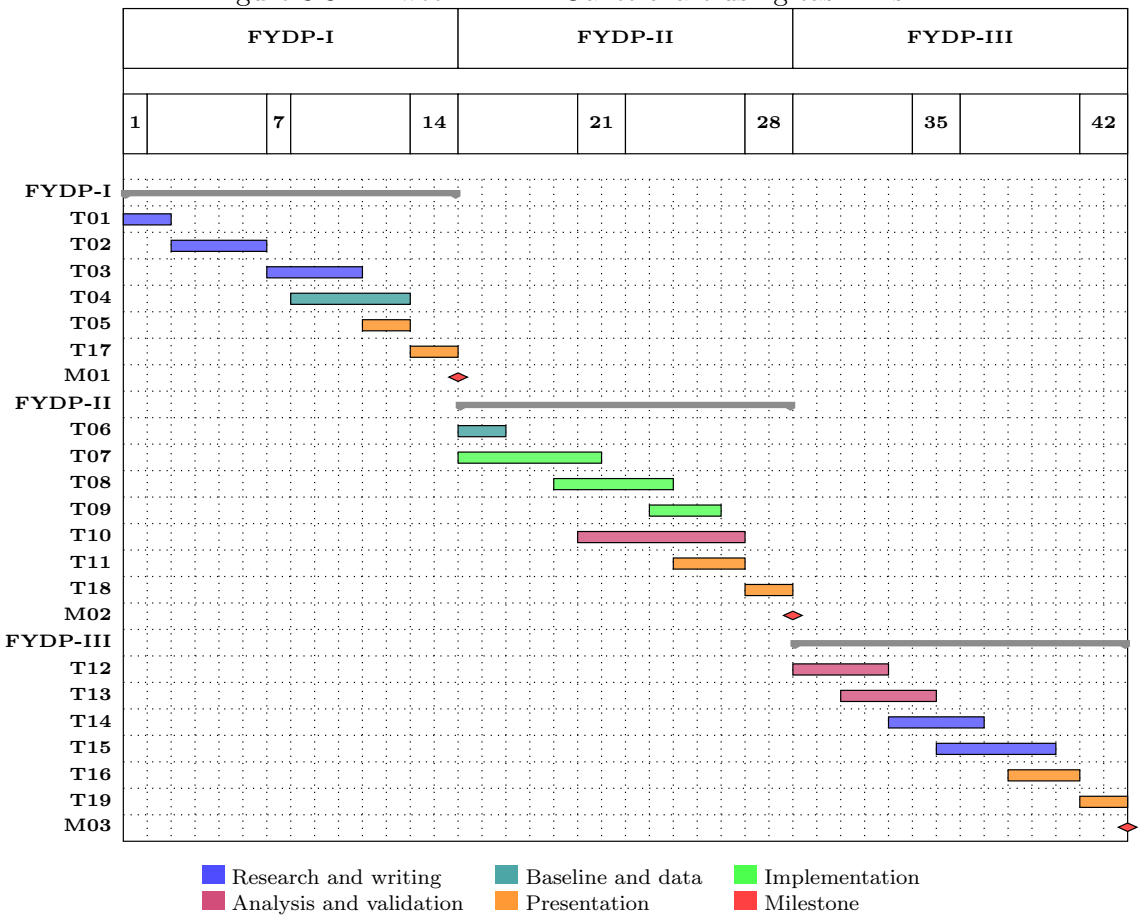

Table 3.5: Task and Milestone IDs Used in the Gantt Chart

ID	Task or Milestone	Weeks	ID	Task or Milestone	Weeks
T01	Problem framing and objectives	1–2	T10	Metrics, logging, and visualizations	20–26
T02	Literature review and gap analysis	3–6	T11	Chapter 4 draft and presentation	24–26
T03	Requirements and standards mapping	7–10	T18	Defense preparation and final presentation	27–28
T04	Baseline validation and dataset specification	8–12	M02	FYDP-II submission milestone	28
T05	Interim report and presentation	11–12	T12	Result verification and reproducibility review	29–32
T17	Defense preparation and final presentation	13–14	T13	Comparative analysis and final figures	31–34
M01	FYDP-I submission milestone	14	T14	Standards, constraints, and impact analysis	33–36
T06	Canonical dataset generation	15–16	T15	Final report revision and conclusion	35–39
T07	ANN training pipeline implementation	15–20	T16	Final presentation and defense preparation	38–40
T08	Phase 1 architecture search	19–23	T19	Defense preparation and final presentation	41–42
T09	Phase 2 optimizer comparison	23–25	M03	Final submission and defense	42

3.4 Task Allocation

There are 5 members in our team. We divided up tasks by skill area and also mapped all of the members' contributions to FYDP-I, FYDP-II and FYDP-III. This prevents treating the write chapters without regard to the rest of the book. All members are involved in the same research pipeline consisting of a variety of planning to final defense with different primary ownership.

Table 3.6: Task Allocation Across Team Members and FYDP Segments

Member	**FYDP-I Responsibilities**	**FYDP-II Responsibilities**	**FYDP-III Responsibilities**
Ikramul Hasan Moral	Lead numerical baseline validation; verify the Lorenz-1960 equation setup; define data-generation requirements.	Implement preprocessing, model definitions, training loop, and experiment runner for the 69 planned runs.	Recheck reproducibility, archive scripts and results, and support technical defense of the implementation.
Md. Abu Bakar	Support requirements analysis and metric selection; review literature related to evaluation criteria.	Build the evaluation pipeline; compute MSE, MAE, maximum error, training time, and inference time.	Consolidate final tables; verify result consistency against the requirements; support analysis discussion.
Samiur Rahman Omlan	Prepare methodology diagrams and initial experimental workflow documentation.	Produce convergence plots, solution trajectory plots, and architecture-comparison figures.	Finalize visual evidence for Chapter 4 and presentation slides; support result interpretation.
Fariha Islam	Lead literature review, gap analysis, citation tracking, and Chapters 1–2 drafting.	Maintain citation consistency while implementation details are added to Chapter 4.	Revise the final report for academic tone, citation coverage, and defense-readiness.
Md. Touhidul Islam	Compile Chapter 3 planning material, task allocation, and interim-report structure.	Coordinate weekly documentation, report updates, and experiment-log summaries.	Compile the final report, Chapter 5 constraints, Chapter 6 conclusion, and final presentation narrative.

All members participate in supervisor meetings, progress journals, internal result reviews, and FYDP presentations. Shared review is necessary because an error in the solver, dataset, model configuration, or interpretation would affect the whole research conclusion.

3.5 Summary

Chapter 1 into a conceptual framework.The research question from Chapter 1 and literature gap from Chapter 1 was transformed into a conceptual framework in this chapter. Chapter 2 into a concrete experimental design. Functional requirements tell you what the pipeline must yield and the non-functional requirements tell you the quality the pipeline must achieve. The context The pipeline is put in its research environment by diagram and Level 1 data flow diagram. A two-phase comparative study methodology is used. A 60-configuration phase 1 run is executed. Search over depth, width and activation function for architecture. Phase 2 is a 9 configuration optimizer comparison of the three best architectures. This means 69 controlled experiments altogether. We calculate reference data and check it on two different solvers. We keep training, loss and evaluation settings

fixed for all the runs so that they are observed . The differences are attributable to variables studied. The project plan assigns the work to the three segments of FYDP (each 14 weeks). There is a two-week period for defense preparation and the final presentation at the end of each phase. The items have been captured as tasks T17, T18 and T19 in the revised 42-week schedule. The task allocation Delegates team members to implementation, analysis and writing. In the next chapter, the implementation of this design and the experimental results from it are reported 69 runs.

Chapter 4

Implementation and Results

Chapter 5

Standards and Design Constraints

5.1 Compliance with the Standards

5.2 Design Constraints

5.3 Cost Analysis

5.4 Complex Engineering Problem

This project is a complex engineering problem. It is not just writing code. We need to understand math, build numerical solvers, design neural networks, and compare results carefully. The Lorenz-1960 system is nonlinear and coupled, so solving it with an ANN requires real engineering decisions, not just running a script.

5.4.1 Complex Problem Solving

Table 5.1 shows how this project maps to the complex problem solving categories.

Table 5.1: Mapping with complex problem solving.

P1 Dept of Knowledge	P2 Range of Conflicting Requirements	P3 Depth of Analysis	P4 Familiarity of Issues	P5 Extent of Applicable Codes	P6 Extent of Stakeholder Involvement	P7 Interdependence
✓	✓	✓	×	×	×	✓

P1: Depth of Knowledge (✓)

Many things are needed to be known at once: how ODEs work, how numerical solvers like Runge-Kutta produces reference solutions, how the neurons learn, how optimizers such as Adam update the weights, and how to measure errors. One of these isn't sufficient. All are linked together.

P2: Range of Conflicting Requirements (✓)

We face trade-offs. A larger network could yield higher accuracy at the cost of training time. and may overfit. A more quickly trained system might be unstable. A model that performs really well the first time around may not be able to run with a different random number seed. We can't have it all at the same time. so, we have to find a good balance.

P3: Depth of Analysis (✓)

We don't simply execute the code and see if it is working. We tested several ANN configurations and observe that: Understand the meaning of the error numbers, observe the behavior of training, determine whether results converge, and why One configuration is better than the other. It's the real analysis.

P4: Familiarity of Issues (×)

This Lorenz-1960 system is a well-known benchmark and the methods we have been using, such as Runge-Kutta solvers, feedforward ANNs, Adam optimizer are all established. Although While our specific combination is new, the issues that are not new to practitioners.Thus this criterion is not applicable.

P5: Extent of Applicable Codes (×)

Our project is a computational research study, not a commercial or safety critical system.We are not currently subject to any industry standards, regulations or professional codes which limit the ways in which we Design and test the accuracy of the ANN. Thus, this criterion is not valid.

P6: Extent of Stakeholder Involvement (×)

Only student researchers and academic supervisors are stakeholders.There are no external clients/end-users/regulatory bodies whose conflicting needs need to be balanced.All decisions are made on the basis of science, not stakeholder demands.So this doesn't applicable for criterion.

P7: Interdependence (✓)

All of the components of this project are interrelated. The solver makes the training data.The data influences the learning process of the ANN. The results are dependent on the ANN design. The optimizer affects training. Just one part goes wrong, the whole result is impacted.

5.4.2 Engineering Activities

Table 5.2 shows how this project maps to the complex engineering activity categories.

Table 5.2: Mapping with complex engineering activities.

A1 Range of resources	A2 Level of Interaction	A3 Innovation	A4 Consequences for society and environment	A5 Familiarity
✓	✓	✓	×	✓

A1: Range of Resources (✓)

We utilise many different tools and sources of knowledge: ODE math, Runge-Kutta solvers, SciPy, PyTorch, NumPy, Matplotlib and 13 research papers. There are no single tools or information source which can handle all this. This is sufficient to complete this project.

A2: Level of Interaction (✓)

This project talk is an on-going dialogue between parts. The data is fed to the ANN by the solver. The ANN design affects training. Training configuration impacts on outcomes. The assessment is based on everything before it. One cannot work on one aspect without taking into consideration the others.

A3: Innovation (✓)

A systematic comparison of the architecture of plain ANNs on Lorenz-1960 system has. This is the first time this system has not been performed. Most of the current works are based on PINNs or hybrid approaches [4]. We pose a new question: is it feasible for a simple ANN with no supervision or feedback to do this? Is it still possible to do a good job on this using physics in the loss function? This is the new section.

A4: Consequences for Society and Environment (×)

Our project is a purely academic study which is training small networks on its own computer.Computer Does not create any physical product, does not impact public health or is

safe and requires little computational resources. The Lorenz-1960 system is a theoretical. We are not benchmarking and are not going to make any real world choices based on our results. This criterion is not applicable.

A5: Familiarity (✓)

This is NOT a textbook type problem. The Lorenz-1960 system is a nonlinear system and chaotic. This is not a pre-made ANN setup that we can just use. We will need to Let's learn by doing, determine the best architecture for our purpose and be careful to understand. when the training loss is low, this doesn't mean that the solution is what it is. is right everywhere.

5.4.3 Knowledge and Attitude Profile

Table 5.3 shows how this project maps to the knowledge and attitude profile categories.

Table 5.3: Mapping with knowledge and attitude profile.

WK1 Natural Sciences	WK2 Math & Analysis	WK3 Eng. Fundamentals	WK4 Specialist Knowledge	WK5 Resource Use	WK6 Eng. Practice	WK7 Role in Society	WK8 Research Lit.	WK9 Ethics
✓	✓	✓	✓	×	✓	×	✓	✓

WK1: Natural Sciences (✓)

We need to have a good knowledge of the natural sciences, particularly physics, in our project. The Lorenz-1960 system is a meteorological model which illustrates atmospheric convection. Understanding of the theory of nonlinear coupled.To accurately model these physical phenomena, ODEs are required.

WK2: Mathematics and Numerical Analysis (✓)

The mathematics and numerical analysis are heavily used in this project and are conceptually based. We use advanced calculus for ODEs, Runge-Kutta method to have reference numerical solutions and statistics to calculate mean squared error of our ANN predictions.The key interest in evaluating training convergence and model accuracy is data analysis.

WK3: Engineering Fundamentals (✓)

We use a structured approach to engineering principles using a clearly defined formulating a problem mathematically and solving it by computer. The translation of the Lorenz-1960 mathematical equations into a computational model is what was called the process.Involves using core engineering methodology.

WK4: Specialist Knowledge (✓)

Our research is on the cutting edge of the field and focuses on the possibilities of artificial application of neural networks to the solution of a complex system of ODEs. This requires specialist engineering knowledge in deep learning models such as PyTorch, optimization algorithms such as Adam, and network architectures.

WK5: Engineering Design (Sustainability) (×)

This project is a theoretical mathematical benchmark study which is computational research. It is not using of physical resources, environmental impact assessments or whole-life cost considerations. Hence this criterion is not applicable.

WK6: Engineering Practice (Technology) (✓)

We extensively use modern engineering practice and technology. We utilize programming languages and scientific computing libraries (SciPy, NumPy, Matplotlib) that are standard practice in the field of computational engineering and data science.

WK7: Role of Engineering in Society (×)

The Lorenz-1960 system benchmark is not directly related to public safety, societal issues, or the use of information technology in the nation. or sustainable development. It's a scholarly exercise of algorithm assessment then, There are no direct societal stakeholders whose safety is to be taken into account. This criterion does not apply.

WK8: Engagement with Research Literature (✓)

To gain insight into how we are expected to act, we did extensive literature review many research papers. The latest advancements in PINNs and conventional ANNs for solving ODEs. We applied No prior studies have explored how the performance of plain ANNs can be assessed on the The Lorenz-1960 system that does not include physics-informed loss functions.

WK9: Ethics and Professional Conduct (✓)

Following the practice of professional ethics and academic integrity, all research is properly cited. For papers and methodologies these were used. Our experiments are reproducible

and our Results are presented truthfully and not manipulated.

5.5 Summary

Chapter 6

Conclusion

The present work investigates whether the same concept of copying the solution of an ordinary differential equations system, as was done for the Lorenz-1960 set, can be applied to the simple feedforward artificial neural network with a much lower accuracy of the reference data, which is in the form of numerical data. Motivation is practical and academic tension. There are classical numerical solvers like Runge-Kutta methods that provide dependable reference solutions, but they solve the system step by step on a fixed grid. A continuous input/output mapping can be learned over time by neural approaches. Although PINNs have been introduced for the solution of differential equations in recent efforts, state variables are still relevant classes of variables. Losses, or operator-learning architectures [4, 5, 6]. This study thus focuses on narrows. The data-driven feedforward ANN is trained to approximate $(x(t), y(t), z(t))$ for the Lorenz-1960 initial value problem, without adding ODE residual terms to the loss function.

The research design is controlled architecture comparison with validated numerical baseline. The parameters for the Lorenz-1960 formulation, initial conditions and time. The system used in the reference problem of Chapter 2 (PinnDE) is used in the intervals. Two numerical solvers are used to produce the reference trajectory and to validate it. The reference trajectory is generated and validated with a custom numerical solver, as well as a commercial numerical solver. RK4 is applied before using it as training data and SciPy DOP853 is used. The ANN study is divided into two parts. In phase 1 the depth, width and optimizer are kept constant and the activation function is changed. During the first phase the depth, width and optimizer are fixed and the activation function is changed. Phase 2 compares Adam, L-BFGS and SGD with momentum on the top-performing phase 1 architectures. This separation is made to make the interpretation of the architecture and optimiser effects easier. The predicted result of the work will be an empirical benchmark to make an approximation of the Lorenz-1960 system to a simple ANN. Previous works have shown that neural networks are able to solve, or to approximate, differential equations of a wide range of formulations [8, 11, 12]. The physics-informed approaches have been used for Lorenz-type problems [4, 18]. The academic merit of this project is that it is based on the simpler and more straightforward question of how far a standard feedforward

ANN can be pushed by systematic variations in its architecture and an exploration of the results it learns. A verified numerical solution is called a training target. The solution can help determine if the bracelet is or is not better. What about the physics informed constraints, is it required for this benchmark or can plain-ANN design be used? These are still necessary to get a higher accuracy and reliability. At this report stage the supported contribution is: The validated baseline design, the dataset and training protocol, and the planned evaluation framework. If the experiment records the declaration of a final best ANN architecture then the report does not state a final best ANN architecture. In Chapter 4, you will find metric tables and plots. This is intentional. Without supporting evidence, any claim of accuracy would detract from the study. Final conclusions regarding the optimal depth, width, activation function or optimizer should be finalized only after careful experimental research and analysis. Planned comparison is completed and research needs to be continued in the study. Three directions. First, the most suitable configuration needs to be tested repeatedly over random Use of seeds to check for stability. Second, in order to assess the generalization of the trained model, it needs to be tested on denser Use time grids or other initial conditions. Third, the ANN results obtained in the plain should be compared with the physics-based, literature informed and operator learning baselines in a more direct manner. These extensions would assist in deciding if the architecture chosen for this project is effective for only one or more of the objects, controlled trajectory or for wider neural approximation of nonlinear ODE systems.

References

[1] Facundo Bre, Juan Gimenez, and Víctor Fachinotti. Prediction of wind pressure coefficients on building surfaces using artificial neural networks. *Energy and Buildings*, 158, 11 2017.

[2] Weida Zhai, Dongwang Tao, and Yuequan Bao. Parameter estimation and modeling of nonlinear dynamical systems based on runge–kutta physics-informed neural network. *Nonlinear Dynamics*, 111:1–14, 10 2023.

[3] Edward N. Lorenz. Maximum simplification of the dynamic equations. *Tellus*, 12(3):243–254, 1960.

[4] Jason Matthews and Alex Bihlo. Pinnde: physics-informed neural networks for solving differential equations. *arXiv preprint arXiv:2408.10011*, 2024.

[5] Liam LH Lau and Denis Werth. Oden: A framework to solve ordinary differential equations using artificial neural networks. *arXiv preprint arXiv:2005.14090*, 2020.

[6] Atmane Babni, Ismail Jamiai, and José Alberto Rodrigues. Error estimates and generalized trial constructions for solving odes using physics-informed neural networks. *Mathematical and Computational Applications*, 30(6):127, 2025.

[7] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.

[8] Sourabh Kumar Dubey, Raghvendra Singh, and Hibah Islahi. Solving ordinary differential equations using artificial neural networks: A comprehensive approach. *Journal of Computational Analysis & Applications*, 33(6), 2024.

[9] Khadeejah James Audu, Olalekan Abdrasheed Kayode, Fajuyi Samuel, Mubarak Inuolaji, and Yahaya Yusuph Amuda. Neural network solutions for first-order differential equations: A physics-informed perspective. *Journal of Engineering and Basic Sciences*, 5:1–13, 2026.

[10] I. E. Lagaris, A. Likas, and D. I. Fotiadis. Artificial neural networks for solving ordinary and partial differential equations. *IEEE Transactions on Neural Networks*, 9(5):987–1000, 1998.

[11] Susmita Mall and Snehashish Chakraverty. Comparison of artificial neural network architecture in solving ordinary differential equations. *Advances in Artificial Neural Systems*, 2013(1):181895, 2013.

[12] Roseline N Okereke, Olaniyi S Maliki, and Ben I Oruh. A novel method for solving ordinary differential equations with artificial neural networks. *Applied Mathematics*, 12(10):900–918, 2021.

[13] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[14] Khadeejah James Audu, Marshal Benjamin, Umar Mohammed, and Yusuph Amuda Yahaya. Utilizing the artificial neural network approach for the resolution of first-order ordinary differential equations. *Malaysian Journal of Science and Advanced Technology*, pages 210–216, 2024.

[15] V Murugesh, M Priyadharshini, Yogesh Kumar Sharma, Umesh Kumar Lilhore, Roobaea Alroobaea, Hamed Alsufyani, Abdullah M Baqasah, and Sarita Simaiya. A novel hybrid framework for efficient higher order ode solvers using neural networks and block methods. *Scientific Reports*, 15(1):8456, 2025.

[16] Danang A Pratama, Maharani A Bakar, NB Ismail, and M Mashuri. Ann-based methods for solving partial differential equations: a survey. *Arab Journal of Basic and Applied Sciences*, 29(1):233–248, 2022.

[17] Xiao Xiong. New designed loss functions to solve ordinary differential equations with artificial neural network. *arXiv preprint arXiv:2301.00636*, 2022.

[18] Muhammad Naeem Aslam, Muhammad Waheed Aslam, Muhammad Sarmad Arshad, Zeeshan Afzal, Murad Khan Hassani, Ahmed M Zidan, and Ali Akgül. Neurocomputing solution for lorenz differential equations through artificial neural networks integrated with pso-nna hybrid meta-heuristic algorithms: a comparative study. *Scientific Reports*, 14(1):7518, 2024.

[19] Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31, 2018.